Data Structures -- Assignment 1

Course:	Data Structures	Course Code: CS 2001	
Program:	BS(SE)	Semester:	FALL-2026
Due Date:	9-Sep-2026	Total Marks:	100
Type:	Assignment 1	Page(s):	15
Course Instructor	Rana Waqas Ali	TA.	Syed Aoun Haider Sherazi

Important Instructions:

1. Submit your .cpp files named as your roll number+QuestionNo+FileNo. Dont submit zip files.
2. You are not allowed to copy solutions from other students. We will check your code for plagiarism using plagiarism checkers. If any sort of cheating is found, negative marks will be given to all students involved.
3. Late submission of your solution is not allowed.
4. All questions / parts must be done keeping in mind the time constraint of that Data Structures or marks will be deducted.
5. In case of any confusion, feel free to discuss with Instructor or TA.
6. Do proper edge cases handling for all questions.

Part 1 -- Recruitment Pipeline

NOTE: This question carries 60 marks. Read every sub-section carefully before you begin implementing.

1. Problem Statement

Imagine you are building a simplified version of LinkedIn's recruitment system.

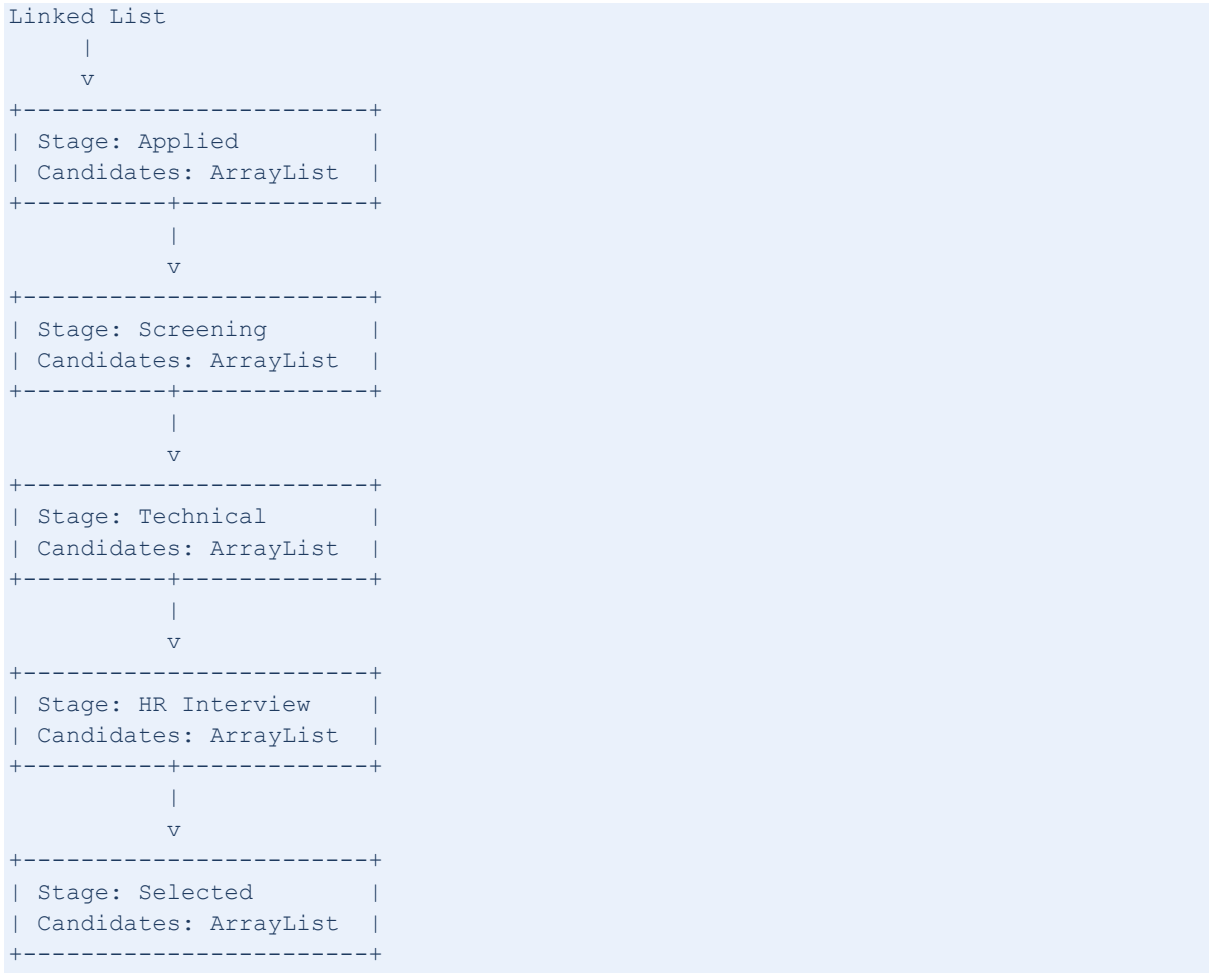
A recruiter is looking for students for an international AI Engineering internship.

Thousands of students have applied, but the recruiter does not want to process everyone at once. Instead, applications are divided into different recruitment stages.

For example:

Applied -> Screening -> Technical Interview -> HR Interview -> Selected

Each stage is represented by a node in a Singly Linked List. However, every stage contains its own ArrayList of candidates:



Your job is to implement this recruitment pipeline. Candidates are constantly moving between stages. Some fail, some get promoted, some withdraw. The recruiter also needs complex pipeline queries. You must design a data structure capable of handling all of this efficiently.

2. Candidate

Every candidate must contain the following information:

- Candidate ID
- Name
- University
- CGPA
- Years of Experience
- Technical Score
- Interview Score
- Skills
- Application Status

For example:

```
ID: 1042
Name: Ali Ahmed
University: FAST
CGPA: 3.72
Experience: 1
Technical Score: 87
Interview Score: 82
Skills: ["C++", "Python", "Machine Learning"]
Status: ACTIVE
```

The Candidate ID must be unique.

3. Recruitment Stage Node

The recruitment pipeline is represented using a Singly Linked List. Each node represents one recruitment stage.

Every node must contain: Stage Name, ArrayList<Candidate>, Next Pointer.

```
HEAD
 |
 v
[Applied | ArrayList<Candidates> | next]
 |
 v
[Screening | ArrayList<Candidates> | next]
 |
 v
[Technical | ArrayList<Candidates> | next]
 |
 v
[HR | ArrayList<Candidates> | next]
 |
 v
[Selected | ArrayList<Candidates> | NULL]
```

RESTRICTION: You are NOT allowed to use another Linked List inside a stage. The candidates inside every stage must be stored using an ArrayList / Dynamic Array.

4. Initial Pipeline

Initially, the recruiter creates the following stages in order:

- Applied
- Screening
- Technical Interview
- HR Interview
- Selected

Your implementation must NOT assume that there will always be exactly five stages. The recruiter may later: add a new stage, remove an existing stage, or insert a stage between two existing stages.

5. Moving Candidates

Implement:

```
moveCandidate(candidateID, destinationStage)
```

The operation must:

1. Locate the candidate.
2. Remove the candidate from their current stage's ArrayList.
3. Insert the candidate into the destination stage's ArrayList.

A candidate must not exist in two stages simultaneously.

6. Candidate Uniqueness and Pipeline Integrity

A candidate must exist in exactly one stage at any given time. Your program must detect duplicate candidates across stages. For example:

```
Applied:   [1001, 1002, 1003]
Screening: [1004, 1005]
Technical: [1003, 1006]
```

Candidate 1003 appears twice. Your program must report:

```
PIPELINE CORRUPTED
Duplicate Candidate: ID = 1003
Found in: Applied, Technical Interview
```

7. Candidate Withdrawal

Implement:

```
withdrawCandidate(candidateID)
```

The candidate should be removed from whichever stage currently contains them.

- If the candidate does not exist: "Candidate not found."
- If they have already withdrawn: "Candidate has already withdrawn."

8. Updating Candidate Scores

Implement:

```
updateTechnicalScore(candidateID, newScore)
updateInterviewScore(candidateID, newScore)
```

A candidate's Technical Score can only be updated while they are in the Technical Interview stage. Similarly, Interview Score can only be updated during the HR Interview stage. Invalid updates must be rejected.

9. Automatic Promotion

Implement:

```
promoteEligibleCandidates()
```

Promotion rules:

Transition	Requirement
Applied -> Screening	CGPA >= 3.0
Screening -> Technical Interview	CGPA >= 3.2 AND at least 2 relevant skills
Technical Interview -> HR Interview	Technical Score >= 70
HR Interview -> Selected	Technical Score >= 80 AND Interview Score >= 75

IMPORTANT: A candidate may move only ONE stage per call to promoteEligibleCandidates(). Do not allow candidates to skip stages during a single call.

10. Best Candidate

```
getBestCandidate()
```

Final Score = 0.40 x CGPA Score + 0.35 x Technical Score + 0.25 x Interview Score

Candidates from all stages must be considered. Tiebreaking:

4. Higher Technical Score wins.
5. If still tied, higher CGPA wins.
6. If still tied, smaller Candidate ID wins.

11. Skill-Based Search

```
findCandidatesBySkill(skill)
```

Return all candidates possessing the requested skill. Results must be presented in current pipeline stage order.

12. Most Crowded Stage

```
getMostCrowdedStage()
```

Return the stage containing the most candidates. If tied, return the earliest stage in the pipeline.

13. Removing a Stage

```
removeStage(stageName)
```

A stage may only be removed if it contains ZERO candidates. Reject the operation otherwise.

14. Inserting a New Stage

```
insertStage(newStageName, afterStageName)
```

Example: insertStage("Online Assessment", "Screening") transforms:

```
Before: Applied -> Screening -> Technical Interview
After:  Applied -> Screening -> Online Assessment -> Technical Interview
```

The newly inserted stage starts with an empty ArrayList.

15. Reversing the Pipeline

```
reversePipeline()
```

```
Before: Applied -> Screening -> Technical -> HR -> Selected
After:  Selected -> HR -> Technical -> Screening -> Applied
```

IMPORTANT: Only the Linked List connections should be reversed. The ArrayList inside every stage must remain unchanged.

16. Detecting a Broken Pipeline

A programming error may cause the Linked List to contain a cycle. Implement:

```
hasCycle()
```

IMPORTANT: Use an in-place cycle-detection algorithm. Do NOT store every visited node in an additional ArrayList/collection.

17. Pipeline Statistics

```
displayStatistics()
```

Display for each stage: number of candidates, relevant average scores, and total candidates in pipeline. Compute dynamically -- do not hard-code any values.

18. Challenging Scenario

Your program must correctly process this sequence of operations on the initial pipeline:

7. updateTechnicalScore(108, 91)
8. moveCandidate(103, "Screening")

9. insertStage("Online Assessment", "Screening")
10. moveCandidate(106, "Online Assessment")
11. promoteEligibleCandidates()
12. withdrawCandidate(110)
13. removeStage("HR")
14. findCandidatesBySkill("C++")
15. getBestCandidate()
16. reversePipeline()
17. displayStatistics()

19. Important Restrictions

Allowed:

- Custom Linked List
- ArrayList / Dynamic Array
- Classes / Structures
- Basic loops, searching, sorting

NOT Allowed:

- STL list
- STL vector (to replace the required ArrayList)
- HashMap / unordered_map for storing the pipeline
- Built-in Linked List classes
- Global candidate storage that bypasses the required structure

20. Required Classes

Your implementation must contain at least:

- Candidate -- stores all candidate information
- Stage -- contains stageName, ArrayList<Candidate>, next pointer
- RecruitmentPipeline -- contains head and implements all major operations

21. Complexity Requirement

For every major function, state its Time Complexity and Space Complexity with a brief justification. This will be graded alongside correctness.

Part 2 -- DSA Problem

Clone a Linked List with Random Pointers

Problem Statement

You are given a special singly linked list where each node contains two pointers:

- next -- points to the next node in the sequence (or NULL for the last node).
- random -- points to ANY arbitrary node in the list, or NULL.

Your task is to construct a DEEP COPY (clone) of this linked list such that:

18. Every node in the clone is a newly allocated node -- no pointer in the clone may reference any node from the original list.
19. The next and random pointers of each cloned node must mirror exactly those of its corresponding original node.
20. The original list must remain completely unmodified after the cloning operation.

Node Definition

```
struct Node {
    int   data;
    Node* next;
    Node* random;  // points to any node in the list, or NULL

    Node(int val) : data(val), next(nullptr), random(nullptr) {}
};
```

Example

Consider a list with 5 nodes (positions 0..4):

```
Node 0: data=1,  next->Node1,  random->Node2
Node 1: data=2,  next->Node2,  random->Node0
Node 2: data=3,  next->Node3,  random->Node4
Node 3: data=4,  next->Node4,  random->Node3
Node 4: data=5,  next->NULL,   random->Node1

Expected Clone:
CloneNode 0: data=1,  next->Clone1,  random->Clone2
CloneNode 1: data=2,  next->Clone2,  random->Clone0
CloneNode 2: data=3,  next->Clone3,  random->Clone4
CloneNode 3: data=4,  next->Clone4,  random->Clone3
CloneNode 4: data=5,  next->NULL,    random->Clone1
```

Constraints

- 0 <= N <= 10^5 (number of nodes)
- -10^4 <= Node.data <= 10^4
- random pointer may be NULL or point to any node (including itself).
- The original list must remain unmodified.

Requirements

21. Implement an efficient solution. An O(N^2) brute-force scan is NOT acceptable.
22. Clearly explain your approach in plain English before writing code.
23. Provide the algorithm / pseudocode.
24. Provide the complete C++ implementation.
25. State the time complexity with justification.
26. State the space complexity with justification.
27. Test on at least 3 distinct examples including: all random=NULL, random pointing to self.

Dynamic Programming is NOT required and NOT expected. Solve using pure Linked List manipulation.

Answer Space

Write your approach, pseudocode, and implementation below (or attach separate sheets):

Part 3 -- Time Complexity Problem Set

This section is provided as printed pages. Solve all four questions on those pages and submit them with your assignment.

For each question:

28. Determine the tight Theta-bound.
29. Show your complete working / derivation.
30. Explain the reasoning used to obtain the answer.

The final answer alone is NOT sufficient. Complete working is mandatory and will be assessed for marks. Ignore print statements, counters, and no-op operations -- focus on the loop structure.

Question 1 --

```
int Sum = 0;

for (int i = 1; i <= N; i++) {

    int count = 0;

    for (int j = 1; i * j <= N; j++) {
        Sum++;
        count++;

        if (count % 50 == 0) {
            cout << "Checkpoint: " << count << endl;
        }
    }

    cout << "Row length: " << count << endl;
}

cout << "Final Sum: " << Sum << endl;
```

Required

Determine the tight Theta-bound of the above code fragment.

Tight Bound: _____

Working (use the space below -- attach extra sheets if needed)

Question 2 --

```
int Sum = 0;

for (int i = 1; i <= N; i++) {

    int count = 0;

    for (int j = 1; j * j <= i; j++) {
        Sum++;
        count++;
    }

    cout << "i = " << i
         << ", count = " << count << endl;
}

for (int i = 0; i < N; i++) {
    Sum += 0;
}

cout << "Final Sum: " << Sum << endl;
```

Required

Determine the tight Theta-bound of the above code fragment.

Tight Bound: _____

Working (use the space below -- attach extra sheets if needed)

Question 3 --

```
int Sum = 0;

for (int i = 1; i <= N; i *= 2) {

    int count = 0;

    for (int j = 1; j <= N; j *= 3) {
        Sum++;
        count++;

        if (count % 5 == 0) {
            cout << "Checkpoint" << endl;
        }
    }

    cout << "Completed block: " << i << endl;
}

cout << "Final Sum: " << Sum << endl;
```

Required

Determine the tight Theta-bound of the above code fragment.

Tight Bound: _____

Working (use the space below -- attach extra sheets if needed)

Question 4 --

```
int Sum = 0;

for (long long i = 2; i <= N; i = i * i) {

    Sum++;

    cout << "Current value: "
         << i << endl;

    if (i > N / 2) {
        cout << "Near termination" << endl;
    }
}

cout << "Final Sum: " << Sum << endl;
```

Required

Determine the tight Theta-bound of the above code fragment.

Tight Bound: _____

Working (use the space below -- attach extra sheets if needed)

General Instructions

Per-Part Instructions

Part 1 -- Recruitment Pipeline

- Read every sub-section carefully before starting your implementation.
- Do not replace the required Singly Linked List + ArrayList structure.
- Handle edge cases: empty pipeline, one-stage pipeline, empty ArrayLists, non-existent candidates, duplicate IDs.
- State Time and Space Complexity for every major function with a brief justification.
- Test with the Challenging Scenario (Section 18) before submitting.

Part 2 -- DSA Problem

- Implement an efficient O(N) solution -- O(N^2) brute force is not accepted.
- Explain your approach in plain English before writing code.
- Provide pseudocode/algorithm alongside your C++ implementation.
- State and justify both time and space complexity.
- Test on at least 3 examples including edge cases (empty list, all random=NULL, random->self).
- Dynamic Programming is not required. Use Linked List manipulation.

Part 3 -- Time Complexity Problem Set

- All four questions must be answered on the provided printed pages.
- Provide: (a) tight Theta-bound, (b) complete working/derivation, (c) brief justification.
- Simply writing the Theta-bound without derivation will not receive full credit.
- Ignore print statements, counters, and no-ops -- focus on the loop structure.
- Attach extra sheets if the provided space is insufficient.

General Rules

- Read all requirements carefully before beginning any implementation.
- Do not hard-code any values -- all outputs must be computed dynamically.
- Complexity answers must be justified; guessed answers will not receive marks.
- Submit neatly organised code with comments explaining your approach.
- Plagiarism will result in zero marks for all parties involved.
- If in doubt about any requirement, contact the TA before the submission deadline.

Submission Checklist

Part	Deliverable	Done
Part 1	Complete C++ source code for the	[ ]

	Recruitment Pipeline with complexity analysis	
Part 2	Approach, pseudocode, implementation, test cases, and complexity for the Clone problem	[ ]
Part 3	Printed pages (or equivalent) with complete working for all 4 time complexity questions	[ ]

Learning Objectives

- Implement and manipulate a Singly Linked List from scratch.
- Combine two data structures (Linked List + ArrayList) to model a real-world system.
- Understand deep copying of non-trivial pointer-based data structures.
- Analyse the time complexity of nested, geometric, and non-linear loops.
- Derive tight Theta-bounds rather than relying on memorised patterns.
- Develop solutions to hard data-structure problems without Dynamic Programming.
- Evaluate both time and space complexity of algorithms.